# Chapter 6 : Functional Dependencies and Normal Forms

# Content

- Functional Dependency
  - Armstrong's axioms
  - Closure
  - Key
  - Key finding algorithm
- Normal Forms
  - Normal form 1
  - Normal form 2
  - Normal form 3
  - Boyce Codd normal form

# 1. Functional dependencies (1)

- Given the relation R,
  two sets of attributes (columns) X and Y on R,
  2 tuples (rows) $r_1$ and $r_2$ on R.
- The functional dependency (FD) *"X determines Y"*, or *"Y depends on X"*, denoted $\mathbf{X} \rightarrow \mathbf{Y}$, is defined as:

  $\forall r_1, r_2 \in R$: If $r_1[X]=r_2[X]$ then $r_1[Y]=r_2[Y]$
  - X: left-hand side (LHS), or determining attributes
  - Y: right-hand side (RHS), or dependent attributes

# 1. Functional dependencies (2)

**For example,** given the following student relationship
STUDENT(Name, Subject, Phone No., Major, Lecturer, Score)

| Name | Subject | Phone No. | Major | Lecturer | Score |
|---|---|---|---|---|---|
| Huy | CSDL | 0913157875 | HTTT | Hưng | 9 |
| Hoàng | CSDL | 0913154521 | HTTT | Hưng | 10 |
| Huy | AV | 0913157875 | HTTT | Thủy | 5 |
| Hải | Toán SXTK | 0166397547 | MạngMT | Lan | 10 |
| Tính | HQTCSDL | 012145475 | CNPM | Sang | 7 |
| Tính | LậpTrình | 012145475 | CNPM | Việt | 8 |
| Hoàng | LậpTrình | 0913154521 | HTTT | Việt | 10 |

# 1. Functional dependencies (3)

**Some properties:**

Each **Name** has a unique **Phone No.** and **Major.**

For each **Subject** there is only one **Lecturer.**

For each **Name, Subject** there is only one **Score.**

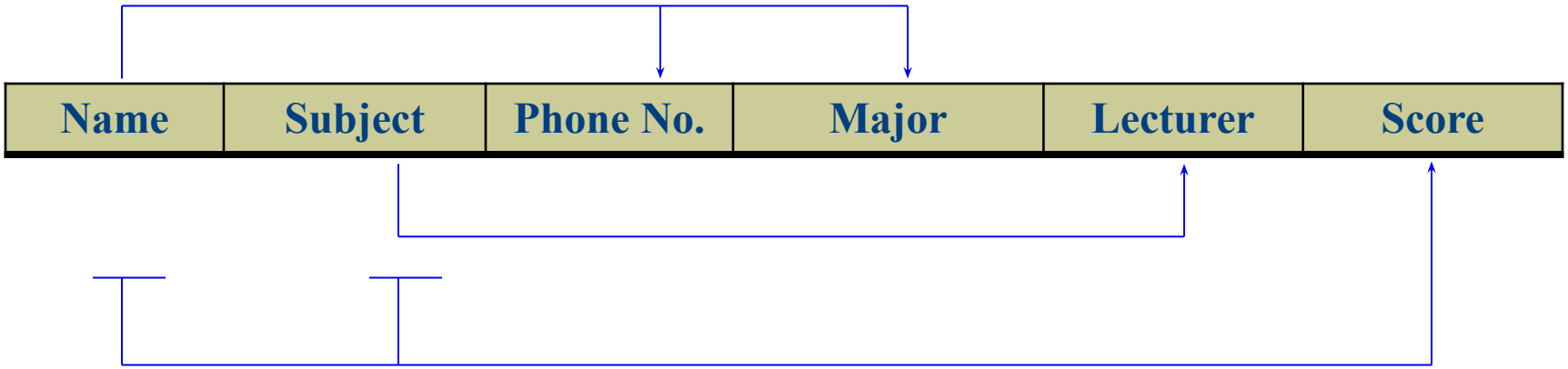
**Expression:**

{Name} → {Phone No., Major}

{Subject} → {Lecturer}

{Name, Subject} → {Score}

# 1. Functional dependencies (4)




The above FDs can infer other functional dependencies:

{Name} → {Major}

{Subject, Score} → {Lecturer, Score}

# 2. Armstrong's axiom system (1)

**Armstrong's axioms are basic rules, assumed to be true, from which other functional dependencies can be inferred.**

Let F be a set of functional dependencies on the relation R.

**Definition** ($F \models X \rightarrow Y$)**:**

F implies $X \rightarrow Y$, or $X \rightarrow Y$ is derived from F, when for any tuple $r \in R$: if r satisfies F then it also satisfies $X \rightarrow Y$.

# 2. Armstrong's axiom system (2)

**Primary rules of Armstrong's axiom system:**

With sets of attributes $X, Y, Z, W \subseteq U$,

**(F1)** Reflexivity: Nếu $Y \subseteq X$ thì $X \rightarrow Y$

**(F2)** Augmentation: $\{X \rightarrow Y\} \models XZ \rightarrow YZ$

**(F3)** Transitivity: $\{X \rightarrow Y, Y \rightarrow Z\} \models X \rightarrow Z$

Secondary rules derived from primary rules for convenience:

**(F4)** Union: $\{X \rightarrow Y, X \rightarrow Z\} \models X \rightarrow YZ$

**(F5)** Composition: $\{X \rightarrow Y, A \rightarrow B\} \models XA \rightarrow YB$

**(F6)** Decomposition: $\{X \rightarrow YZ\} \models \{X \rightarrow Y, X \rightarrow Z\}$

**(F7)** Pseudo-Transitivity: $\{X \rightarrow Y, YZ \rightarrow W\} \models XZ \rightarrow W$

# 2. Armstrong's axiom system (3)

**For example:** $F = \{A \rightarrow B, A \rightarrow C, BC \rightarrow D\}$, prove $A \rightarrow D$?

1) $A \rightarrow B$
2) $A \rightarrow C$
3) $A \rightarrow BC$ (from 1 and 2, Union F4)
4) $BC \rightarrow D$
5) $A \rightarrow D$ (from 3 and 4, Transitivity F3)

# 2. Armstrong's axiom system (4)

**For example:** F = { A → C, B → D}, prove AB → ABCD?

1) AB → A (Reflexivity F1)
2) AB → B (Reflexivity F1)
3) A → C
4) AB → C (from 1 and 3, Transitivity F3)
5) B → D
6) AB → D (from 2 and 5, Transitivity F3)
7) AB → ABCD (from 1, 2, 4, 6, Union F4 )

# 3. Closure (1)

**Closure of a set of functional dependencies**

The closure of a set of functional dependencies F, denoted $F^+$, is the set of all functional dependencies that can be inferred from F.

If $F = F^+$, then F is a complete set of functional dependencies.

**Closure of a set of attributes**

The closure of a set of attributes X with respect to a set of functional dependencies F, denoted by $X^+_F$, is the set of all attributes A that can be functionally determined by X using $F^+$:

$$X^+_F = \{ A \in Q^+ \mid X \rightarrow A \in F^+ \}$$

# 3. Closure (2)
# Closure finding algorithm

**Input:** Relation Q with set of attributes $Q^+$, set of functional dependencies F on Q, set of input attributes $X \subseteq Q^+$

**Output:** Attribute closure $X^+_F$

**Algorithm:** Find the sequence $X_{(0)}, X_{(1)}, \ldots, X_{(i)}$ such that $X_{(i)} = X^+_F$

*Initialization:* $X_{(0)} = X$

*Iteration:* For each functional dependency $Y \rightarrow Z$ in F:

If $Y \subseteq X_{(i-1)}$ then: $X_{(i)} = X_{(i-1)} \cup Z$

Remove $Y \rightarrow Z$ from F

Stop when $X_{(i)} = Q^+$ **or** $X_{(i)}$ does not change after an iteration

*Conclusion:* $X^+_F = X_{(i)}$

# 3. Closure (3)

**For example:**

Given the relation diagram R(A, B, C, D, E, G, H) and the set of functional dependencies F={f1: B → A, f2: DA → CE, f3: D → H, f4: GH → C, f5: AC → D}

Find $AC^{+}_{F}$ ?

# 3. Closure (4)

**Step 1:** $X_{(0)} = AC$

**Step 2:** f1 to f4 are not satisfied,

f5 satisfies so $X_{(1)} = AC \cup D = ACD$

Repeat step 2: (f5 already satisfied)

f1 not satisfied,

f2 satisfies so $X_{(2)} = ACD \cup CE = ACDE$

f3 satisfies so $X_{(3)} = ACDE \cup H = ACDEH$

f4 not satisfied

Repeat step 2: (f2, f3, f5 already satisfied)

f1 and f4 are not satisfied.

So $X_{(3)} = ACDEH$ does not change. Stop.

So $AC^{+}_{F} = ACDEH$

# 3. Closure (5)

**For example:**

Given the relational schema R(A, B, C, D, E, G) and functional dependencies F={f1: AB → C, f2: D → EG, f3: ACD → B, f4: C → A, f5: BE → C, f6: CE → AG, f7: BC → D, f8: CG → BD}

Find $BD^{+}_{F}$ ?

*Result: ABCDEG*

# 3. Closure (6)

**The membership problem**

Given a relation Q, a set of functional dependencies F on Q and a functional dependency $X \rightarrow Y$ on Q. The question is whether $X \rightarrow Y \in F^+$ or not?

$$\mathbf{X \rightarrow Y \in F^+ \Leftrightarrow Y \subseteq X^+_F}$$

**For example:**

From the example, find the closure of the attribute set AC. Given that $AC \rightarrow E$ belongs to $F^+$?

We have $AC^+_F$=ACDEH

Because $E \in AC^+_F$ should $AC \rightarrow E \in F^+$

# 4. Key

**Definition:**

Given a relation Q with a set of attributes $Q^+ = \{ A1, A2, \ldots, An \}$, F is a set of functional dependencies on Q, K is a subset of $Q^+$.

Then K is called a key of Q if:

(i) $K^+_F = Q^+$

(ii) There does not exist $K' \subset K$ *such that* $K'^+_F = Q^+$

*(that is, K is the smallest subset satisfying (i) )*

- The set of attributes K" is called a ***super key*** if $K \subseteq K''$.
- Attribute A is called a ***key attribute*** if $A \in K$ . Otherwise attribute A is called a non-key attribute.

# 4. Key finding algorithm (1)

**(Key finding algorithm using dependency graph )**

**Step 1: Build the dependency graph**

- Each node of the graph represents an attribute of the relation R
- Connections represent functional dependencies (many ways to represent)

**Step 2: Find the root node, leaf node, and intermediate node**

- Root node: only outgoing arrows, or isolated nodes
- Leaf node: only arrows go in
- Intermediate buttons: the remaining buttons, including in and out arrows

**Step 3: Calculate closure and find key**

- Start with X = set of root nodes. *(Key must contain at least one root node.)*
- Calculate the closure $X^+_F$.
- If $X^+_F = Q^+$ then X is a key. Stop.
- Otherwise, $X^+_F \neq Q^+$, then add each intermediate node to X in turn.
- Repeat calculating closure $X^+_F$. If the key is found, stop.

# 4. Key finding algorithm ( 2 )

**Tips:** Graph is not required, functional dependencies can be used directly.

**Practical way to find root node, leaf node, intermediate node:**

**- Root nodes N:** only on the left side, or not in any functional dependency.

- List all attributes
- Remove any attributes that appear on the right side.

**- Leaf nodes L:** only on the right side of functional dependencies.

- List the attributes that appear on the right side (potential leaves)
- Remove any attributes that appear on the left side.

**- Intermediate nodes T:** the remaining nodes, on both the left and right.

- The remaining attributes, after removing the root and leaf nodes

# 4. Key finding algorithm ( 3 )

**For example:**

Given the relational schema R(A, B, C, D, E, G, H) and the set of functional dependencies F={B → A , DA → CE, D → H, GH → C, AC → D}

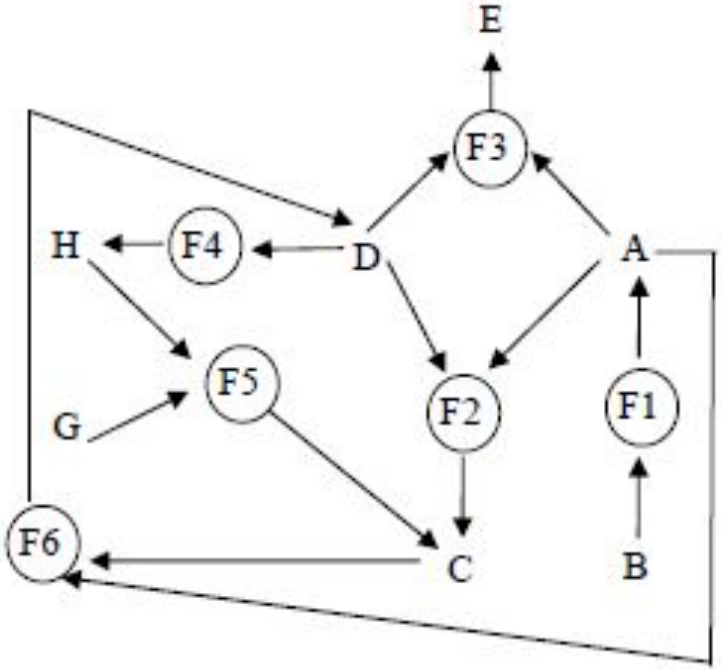
Looking for a key of R?

# 4. Key finding algorithm ( 4 )

- Decompose the RHS of the functional dependencies:

F ={F1: B → A , F2: DA → C,
F3: DA → E, F4: D → H,
F5: GH → C, F6: AC → D}

- Represent F as a graph:




➔ Root nodes N: { B , G}
➔ Leaf nodes L: {E}
➔ Intermediate nodes T: {A, C, D, H}

# 4. Key finding algorithm ( 5 )

- $BG^{+}_{F} = ABG \neq Q^{+}$

  $\rightarrow$ BG is not a key.

  $\rightarrow$ Add attribute D.
- $BDG^{+}_{F} = BDGACEH = Q^{+}$

  $\rightarrow$ BDG is a key.
- Besides, there may be other keys.

# 4. Key finding algorithm ( 6 )

**Example 2:**

Given the relation R(A, B, C, D, E, G)

and the set of functional dependencies F={ AE → C , CG → A, BD → G, GA → E}

Find all keys of R?

***Hint:***

- *Try union with each subset (non-empty) of intermediate nodes.*
- *Stop checking any case that contains any known key.*
- *Avoid duplication when listing subsets.*

Result: ABD, BDC

# 5. Normal Forms

- Normal form 1
- Normal form 2
- Normal form 3
- Boyce Codd normal form

# 5. First Normal Form (1NF) (1)

**The relation Q is in first normal form (1NF) if :**

**Every attribute has an atomic value.**

- An atomic value is a value that cannot be further divided.
- *Multi-valued* and *composite* attributes are not atomic.

# 5. First Normal Form (1NF) (2)

**Example: composite attribute**

*Address = "No. 175, 3/2 Street, Saigon Ward"*

Composite: Address → (House Number, Street, Ward)

⇒ Not atomic value. ⇒ Not in 1NF.

# 5. First Normal Form (1NF) (3)

**Example: composite and multivalued attributes**

| MaHD | MaKH | NgayHD | CtietMua | | | SoTien |
|---|---|---|---|---|---|---|
| | | | Tên hàng | Số lượng | ĐVT | |
| HD01 | KH01 | 15-10-05 | Bánh Orion | 1 | Gói | 25.000 |
| | | | Kẹo mút | 2 | Cây | 2.000 |
| HD02 | KH01 | 18-10-05 | Gạo | 2 | Kg | 30.000 |
| HD03 | KH02 | 24-10-05 | Đường | 1 | Kg | 15.000 |
| | | | Bánh AFC | 2 | Gói | 24.000 |

**CtietMua** is both composite (Product name, Quantity, Unit) and multi-valued ⇒ not atomic ⇒ The relation is not in 1NF.

# 5. Second Normal Form (2NF) (1)

**The relation Q is in second normal form (2NF) if:**

**(1) Q is already in 1NF.**

**(2) Every non-key attribute of Q is *fully dependent* on the key.**

***Full dependency*** on the key means:

- Each non-key attribute depends on ***the entire key***.
- That is, no subset of the key determines that attribute.

# 5. Second Normal Form (2NF) (2)

**To test 2NF:**

**Step 1:** Find all keys of Q.

**Step 2:** For each key K, for each proper subset $S_i$ of K, find the closure $S_i^+$.

**Step 3:** If there is any closure $S_i^+$ containing a non-key attribute then Q is not in 2NF.

# 5. Second Normal Form (2NF) (3)

**Example:**

Given Q1 (A, B, C, D), F={A→B, B→DC}

The schema has only one key, A, so all attributes are fully dependent on the key.

⇒ Therefore, Q1 is in 2NF.

**Example:**

For Q2 (A, B, C, D), F={AB → D, C → D}

The scheme has key ABC. There is C⊂ABC such that C → D, where D is a non-key attribute (i.e. attribute D is not fully dependent on the key).

⇒ Therefore, Q2 is not in 2NF.

# 5. Second Normal Form (2NF) (4)

**Example:**

Let Q3(A,B,C,D,E,G), F={BG→D, B→E , DE→C , CD→A}

The schema has key BG. There is B⊂BG such that B→E, where E is a non-key attribute (i.e. attribute E does not fully depend on the key).

⇒ Therefore, Q2 is not in 2NF.

**Example:**

For Q4 (A, B, C, D), F={AB → C, B → D, BC → A}

The schema has keys AB, BC. There is B⊂AB such that B→ D, where D is a non-key attribute (i.e. D does not fully depend on the key).

⇒ Therefore, Q2 is not in 2NF.

# 5. Third Normal Form (3NF) (1)

**The relation Q is in third normal form (3NF) if:**

- **Q is already in 2NF.**
- **Every non-key attribute of Q is not transitively dependent on the key of Q.**

Or:

**A relation Q is in 3NF if every non-trivial functional dependency $X \rightarrow A$ ( i.e. $X \rightarrow A \in F^+$ on Q , with $A \notin X$) has:**

**(1) X is a super key (left side contains the key)**

**or**

**(2) A is the key attribute (right side is a subset of the key)**

# 5. Third Normal Form (3NF) (2)

**To test 3NF:**

**Step 1:** Find all keys of Q.

**Step 2:** Decompose the right-hand side of every functional dependency in F so that the right-hand side is single-attribute.

**Step 3:** If every functional dependency $X \rightarrow A \in F$, $A \notin X$, satisfies

(1) X is a super key (left side contains the key)

**or**

(2) A is the key attribute (right side is a subset of the key)

then Q is in 3NF, otherwise Q is not in 3NF.

# 5. Third Normal Form (3NF) (3)

**Example:**

Given Q (A, B, C, D), F={AB → D, C → D}

**Step 1:** Q has a key ABC.

**Step 2:** Every functional dependency in F has a single attribute on the right side.

**Step 3:** Consider AB → D, non-trivial, we have:

- The left side (AB) is not a super key.
- Furthermore, the right side (D) is not a key attribute.

⇒ So Q is not in 3NF.

# 5. Third Normal Form (3NF) (4)

**Example:**

Given Q (A, B, C, D, G, H), F={AB → C, AB → D, AB → GH}

**Step 1:** Q has a key AB.

**Step 2:** Every functional dependency in F has a single attribute on the right side.

**Step 3:** Consider all functional dependencies in F, we have:

The left side (AB) is a super key.

⇒ So Q is in 3NF.

# 5. Third Normal Form (3NF) (5)

**Example:**

Given Q (A, B, C, D, E, G), F={A→ BC, AB →D, AC →E, B → G}

**Step 1:** Q has a key A.

**Step 2:** Decompose the right-hand side into single attributes.
F={A→ B, A→ C, AB →D, AC →E, B → G}.

**Step 3:** We have functional dependencies A→B and B →G:

Thus, the non-key attribute G depends transitively on the key A.

⇒ So Q is not in 3rd normal form.

# 5. Third Normal Form (3NF) (6)

**Example:**

Given Q (A, B, C, D), F={AB→ C, D→B, C →ABD}

**Step 1:** Q has keys {AB, AD, C}.

**Step 2:** Decompose the right-hand side into single attributes.

F={AB→ C, D→B, C →A, C →B, C →D}.

**Step 3:** Consider everything functional dependencies in F, we have:

The right side is the key attribute .

⇒ So Q is in 3rd normal form.

# 5. Boyce Codd Normal Form (BCNF) (1)

**The relation Q is in the BCNF if:**

**Every non-trivial functional dependency $X \rightarrow A \in F^+$, with $A \notin X$, has:**

- **X as a superkey (left side contains the key).**

*Note: BCNF is stricter than 3NF.*

# 5. Boyce Codd Normal Form (BCNF) (2)

**To test BCNF:**

**Step 1:** Find all keys of Q.

**Step 2:** Decompose the right-hand side of every functional dependency in F so that the right-hand side is single-attribute.

**Step 3:** If every functional dependency $X \rightarrow A \in F$, $A \notin X$, satisfies

X is a super key (left side contains the key)

then Q is in BCNF, otherwise Q is not in BCNF.

# 5. Boyce Codd Normal Form (BCNF) (3)

**Example:**

Given Q (A, B, C, D, E, I), F={ACD → EBI, CE → AD}

**Step 1:** Q has two keys {ACD, CE}

**Step 2:** Decompose the right-hand side into single attributes.

F={ACD → E, ACD → B, ACD → I, CE → A, CE → D}

**Step 3:** Consider all functional dependencies in F, we have:

The left-hand side is a super key.

⇒ So Q is in BCNF.

# 5. Boyce Codd Normal Form (BCNF) (4)

**Example:**

Given Q (A, B, C, D,G,H), F={AB → C, AB → D, AB → GH}

**Step 1:** Q has a key AB

**Step 2:** Decompose the right-hand side into single attributes.

F={AB → C, AB → D, AB → G, AB → H}

**Step 3:** Consider all functional dependencies in F, we have:

The left-hand side (AB) is a super key.

⇒ So Q is in BCNF.

# 5. Boyce Codd Normal Form (BCNF) (5)

**Example:**

Given Q (A, B, C, D,G,H), F={AB → C, AB → D, AB → GH, B → H}

**Step 1:** Q has a key AB.

**Step 2:** Decompose the right-hand side into single attributes. F={AB → C, AB → D, AB → G, AB → H, B → H}.

**Step 3:** Consider the functional dependency B → H , we have:

The left-hand side (B) is not a super key.

⇒ So Q is not in BCNF.